Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту



Звіт

про виконання лабораторних та практичних робіт блоку № 3
На тему: «Цикли. Вкладені Цикли. Завершення виконання циклів. Функції. Рекурсія. Вбудовані функції. Одновимірні масиви.»
з дисципліни: «Основи програмування»
до:
Лабораторної Роботи № 2
Лабораторної Роботи № 3
Практичних Робіт до блоку № 3

Виконала:
Студентка групи ШІ-13
Сєрова Дарія Віталіївна

Львів 2026

Тема роботи:

Основи та розширені можливості мови C++: керування потоком виконання (цикли), структури даних (одновимірні масиви) та модульність коду (функції, рекурсія, вбудовані функції).

Мета роботи:

Ознайомитися з синтаксисом і принципами використання циклічних конструкцій, одновимірних масивів, функцій, механізмів рекурсії та вбудованих функцій C++; засвоїти алгоритми обробки масивів (пошук, сортування), способи керування циклами та передачі даних у функції; навчитися проектувати, реалізовувати й оптимізовувати модульний код, будувати блок-схеми, оцінювати складність задач та оформлювати й публікувати результати роботи на GitHub відповідно до вимог ітерації.

Теоретичні відомості:

1. Теоретичні відомості з переліком важливих тем:

- Тема №1: Введення в Цикли та їх Види в C++
- Тема №2: Управління Виконанням Циклів
- Тема №3: Вкладені Цикли
- Тема №4: Одновимірні Масиви
- Тема №5: Алгоритми обробки та робота з одновимірними масивами
- Тема №6: Основи Функцій у C++
- Тема №7: Розширені Можливості Функцій
- Тема №8: Вбудовані Функції в C++

2. Індивідуальний план опрацювання теорії:

- Тема №1: Введення в Цикли та їх Види в C++
    - Джерела Інформації
        - Практичні заняття з Основ Програмування
        - Лекції О. Пшеничного
        - W3Schools — C++ While Loop & For Loop
    - Що опрацьовано:
        - Вивчено значення циклічних процесів у програмуванні та синтаксис основних операторів повторення в C++: циклу з передумовою while, циклу з постумовою do-while та параметризованого циклу for. Опрацьовано базові сценарії використання кожного типу циклу залежно від умов задачі.
    - Статус: Ознайомлена

- Початок опрацювання теми: 10.10.26
- Звершення опрацювання теми: 12.10.26

- Тема №2: Управління Виконанням Циклів
  - Джерела Інформації:
    - Лекції О. Пшеничного
    - W3Schools — C++ Break and Continue
  - Що опрацьовано:
    - Ознайомлена з механізмами керування перебігом виконання циклів за допомогою операторів break (переривання) та continue (перехід до наступної ітерації). Засвоєно методи запобігання зациклюванню та правила проектування умов дострокового завершення циклічних процесів.
  - Статус: Ознайомлена
  - Початок опрацювання теми: 10.10.26
  - Звершення опрацювання теми: 12.10.26

- Тема №3: Вкладені Цикли
  - Джерела Інформації:
    - Лекції О. Пшеничного
    - Практичні заняття
    - W3Schools — C++ Nested Loops
  - Що опрацьовано:
    - Опрацьовано концепцію вкладеності циклів для розв'язання багатовімірних та комбінаторних задач (робота з матрицями, побудова таблиць, форматоване виведення даних). Засвоєно особливості зміни лічильників у зовнішніх та внутрішніх циклах та їх часову складність.
  - Статус: Ознайомлена
  - Початок опрацювання теми: 10.10.26
  - Звершення опрацювання теми: 12.10.26
- Тема №4 Одновимірні Масиви
  - Джерела Інформації:
    - Практичні заняття.
    - W3Schools — C++ Arrays
  - Що опрацьовано:
    - Вивчено основи роботи з одновимірними статичними масивами в C++: оголошення, ініціалізація, розташування

елементів у пам'яті, індексація та доступ до елементів за допомогою циклів (for, range-based for).

- Статус: Ознайомлена
- Початок опрацювання теми: 10.10.26
- Звершення опрацювання теми: 12.10.26

- Тема №5 Алгоритми обробки та робота з одновимірними масивами
  - Джерела Інформації:
    - Практичні та лабораторні заняття
    - W3Schools — C++ Loop Through an Array
  - Що опрацьовано:
    - Засвоєно базові алгоритми обробки масивів: знаходження мінімального/максимального елемента, обчислення суми та середнього значення, алгоритми пошуку (лінійний пошук) та класичні алгоритми сортування (наприклад, сортування бульбашкою або вибором).
  - Статус: Ознайомлена
  - Початок опрацювання теми: 10.10.26
  - Звершення опрацювання теми: 27.10.2026

- Тема №6 Основи Функцій у C++
  - Джерела Інформації:
    - Лекції Пшеничного
    - W3Schools — C++ Functions
  - Що опрацьовано:
    - Ознайомлена з принципами модульного програмування. Засвоєно синтаксис прототипів та визначень функцій, повернення значень через return, а також відмінності між передачею аргументів за значенням (by value) та за посиланням (by reference).
  - Статус: Ознайомлена
  - Початок опрацювання теми: 10.10.26
  - Звершення опрацювання теми: 27.10.2026
- Тема №7 Розширені Можливості Функцій
  - Джерела Інформації:
    - W3Schools — C++ Recursion
  - Що опрацьовано:
    - Вивчено розширені механізми функцій: специфікатори області видимості та часу життя змінних (static, extern), виклики функцій

за замовчуванням, поняття та принципи побудови рекурсивних функцій (базовий випадок, крок рекурсії), а також особливості передачі масивів у якості параметрів функцій.

- Статус: Ознайомлена
- Початок опрацювання теми: 10.10.26
- Звершення опрацювання теми: 27.10.2026

- Тема №8 Вбудовані Функції в C++
  - Джерела Інформації:
    - W3Schools — C++ Math Functions
  - Що опрацьовано:
    - Опрацьовано використання специфікатора inline для оптимізації продуктивності, а також застосування стандартних вбудованих функцій C++ із бібліотек <cmath> (математичні обчислення), <cstdlib> та <algorithm> для спрощення та скорочення коду.
  - Статус: Ознайомлена
  - Початок опрацювання теми: 10.10.26
  - Звершення опрацювання теми: 27.10.2026

Виконання роботи:

1. Опрацювання завдання та вимог до програм та середовища:

Завдання №1 Lab# programming: Lab 2
Варіант завдання: 13

13) Знайти суму ряду з точністю ε=0.0001, загальний член якого

$$a_n = \frac{3^n n!}{(3n)!}$$

Рис 1. Умова задачі lab 2

Деталі завдання:

1. При визначенні суми членів ряду варто використовувати рекурентну формулу для отримання наступного члена ряду.
2. При складанні програми вважати, що точність досягнута, якщо an<ε

Завдання №2 Lab# programming: Lab 3
Варіант завдання: 9

Для x, що змінюється від a до b з кроком (b-a)/k, де (k=10), обчислити функцію f(x), використовуючи її розклад в степеневий ряд у двох випадках:
а) для заданого n;
б) для заданої точності ε (ε=0.0001).
Для порівняння знайти точне значення функції.

9	$y=\frac{1}{4}\ln\frac{1+x}{1-x}+$ $+\frac{1}{2}arctgX$	$0{,}1\le x\le 0{,}8$	3	$S=x+\frac{x^5}{5}+.....+\frac{x^{4n+1}}{4n+1}$

Рис 2. Умова задачі lab 3

Деталі завдання:

1. Алгоритм розв'язання завдання зводиться до трьох циклів, причому два з них вкладені в третій. Внутрішні цикли підсумують доданки при фіксованому параметрі x, один (арифметичний для заданого n), інший (ітераційний для заданої точності ε. При організації цих циклів варто звернути увагу на правильний вибір формули для обчислення елемента ряду a_n і правильне присвоєння початкових значень змінним циклу. Зовнішній цикл організує зміну параметра x.
2. Результати розрахунків надрукувати у такому вигляді:

Обчислення функції

X=..... SN=..... SE=..... Y=.....
X=..... SN=..... SE=..... Y=.....
.....
X=..... SN=..... SE=..... Y=.....

Тут X- значення параметра; SN- значення суми для заданого n; SE- значення суми для заданої точності; Y-точне значення функції.

Рис 3. деалі заввдання lab 3

Завдання №3 Practice# programming: Class Practice Task

Аналізатор місячного бюджету

Задача

Ви пишете програму обліку особистих витрат. Користувач задає бюджет на місяць і вводить суму витрат за кожен день.

Далі програма аналізує ці дані через меню: показує статистику, знаходить найдорожчі дні й перевіряє, чи вистачає бюджету.

Дні без покупок теж бувають. У різних обчисленнях вони враховуються по-різному.

Обмеження: сторонні бібліотеки використовувати заборонено.

Вхідні дані

Програма спочатку зчитує бюджет і витрати, і лише після цього показує меню:

№	Значення	Позн.	Формат вводу
1	Бюджет на місяць, грн	`budget`	Число, напр. `8000`
2	Кількість днів обліку	`N`	Ціле число, напр. `10`
3	Витрати по днях, грн	`spending[]`	`N` чисел поспіль, напр. `450 0 1200 320 …`

Витрати зберігаються в одновимірному масиві. Облік ведеться не довше одного місяця.

Меню

Меню повторюється, доки користувач не вибере вихід.

Пункт	Операція
`1`	Показати витрати по днях
`2`	Статистика витрат
`3`	Дні з перевищенням денного ліміту
`4`	Три найдорожчі дні
`0`	Вийти

Деталі операцій

1. Показати витрати по днях

- Таблиця: номер дня і сума витрат за цей день.
- Дні нумеруються з `1`, індексація масиву починається з нуля.

2. Статистика витрат

- Мінімальні й максимальні витрати за день. Обидва значення повертає одна функція через параметри-посилання.
- Сумарні витрати. Обчислити рекурсивною функцією, без циклу.
- Залишок бюджету. Якщо витрати більші за бюджет, показати розмір перевитрати як модуль від'ємного залишку, узявши вбудовану математичну функцію.
- Середні витрати за дні з покупками. Дні з нульовими витратами пропускати оператором `continue`.

- День, коли бюджет вичерпано. Циклом `while` накопичувати витрати день за днем і вийти оператором `break`, щойно сума перевищить бюджет. Якщо бюджету вистачило на всі дні, показати залишок.

3. Дні з перевищенням денного ліміту

- Користувач вводить денний ліміт. Якщо ввів `0`, береться значення за замовчуванням `1000` грн.
- Ліміт задається як параметр за замовчуванням.
- Вивести номери днів з перевищенням і їхню кількість.

4. Три найдорожчі дні

- Відсортувати копію масиву за спаданням вкладеними циклами. Готові функції сортування використовувати не можна.
- Вивести три найбільші суми. Якщо днів менше трьох, вивести скільки є.
- Початковий масив має залишитися незмінним.

Вимоги

- `for`: вивести список витрат по днях, обходячи масив.
- `while`: накопичувати витрати день за днем, доки не буде вичерпано бюджет.
- `break`: зупинити цей цикл одразу на дні, коли бюджет вичерпано, а не після перебору всіх днів.
- `do while`: після кожної операції повертати користувача в меню і працювати, доки він не вибере вихід.
- `continue`: пропускати дні без покупок при обчисленні середніх витрат.
- Вкладені цикли: самостійно відсортувати копію масиву для трьох найдорожчих днів, без готових функцій сортування.
- Функції: кожен пункт меню окремою функцією. Одна з них повертає мінімум і максимум через параметри-посилання, інша має параметр за замовчуванням (денний ліміт `1000` грн). Для модуля перевитрати взяти вбудовану математичну функцію.
- Рекурсія: сумарні витрати обчислити рекурсивною функцією, без циклу.
- `goto`: якщо користувач вводить неправильний пункт меню або нечислові символи, перенаправити його назад до головного меню через `goto`. Зациклення не повинно бути.
- Валідація: перевіряти всі вхідні дані, правила визначити самостійно на основі попередній задач.

Завдання №4 Practice# programming: Self Practice Task 1

Апельсини

Обмеження: 2 сек., 256 МіБ

Діти міряються різними речима. У нашій задачі — апельсинами. Марічка і Софійка міряються з Петриком.

Потрібно визначити, чи Марічка й Софійка разом мають більше апельсинів, ніж Петрик.

Вхідні дані

У першому і єдиному рядку задано три цілі числа a, b та c — кількість апельсинів у Марічки, Софійки і Петрика відповідно.

Вихідні дані

У єдиному рядку виведіть відповідь до задачі — `YES`, якщо дівчата разом мають більше апельсинів, ніж Петрик, або `NO` в іншому випадку.

Обмеження

$1 \le a, b, c \le 10^9$.

Рис 4. Умова задачі self practice task 1

Завдання №5 Practice# programming: Self Practice Task 2

Цікава гра

Обмеження: 2 сек., 256 МіБ

Мале Бісеня та Дракон полюбляють проводити дозвілля разом. Сьогодні вони грають в одну дуже цікаву гру.

У них є дошка, що складається з n рядків та m стовпців, всі клітинки якої білі.

Гравці по черзі вибирають одну білу клітинку та зафарбовують її в чорний колір. Бісеня ходить першим. Гравець, який не може зробити хід, тобто на початку ходу якого вся дошка чорна, програє.

Погостривши зубки, Бісеня зрозуміло, що у Дракона велика перевага, адже він двоголовий, а, як то кажуть, «одна голова добре, а дві — краще». Тому воно просить вас допомогти. Вам потрібно сказати за заданими n та m, хто виграє у цій напруженій грі.

Вхідні дані

У єдиному рядку задані два цілих числа n та m — розміри дошки.

Вихідні дані

Єдине слово — `Imp`, якщо переможе Бісеня, та `Dragon`, якщо переможе Дракон.

Обмеження

$1 \le n, m \le 100$.

Рис 5. Умова задачі self practice task 2

Завдання №6 Practice# programming: Self Practice Task 3

Максимум

Обмеження: 2 сек., 256 МіБ

Написати програму, яка знаходить значення найбільшого елемента масиву.

Вхідні дані

У першому рядку задано одне натуральне число n — розмір масиву.

У другому рядку задано n цілих чисел — елементи масиву.

Вихідні дані

У єдиному рядку виведіть одне ціле число — значення найбільшого елемента масиву.

Обмеження

$0 < n < 1000$,

$-12345678 \le$елемент масиву≤ 12345678.

Рис 6. Умова задачі self practice task 3

Завдання №7 Result Documentation Report and Outcomes Placement Activities (Docs and Programs on GitHub)

- Деталі завдання: оформити звіт стосовно проробленої роботи та поділитися ним і кодами програм через Github використовуючи git команди.

Завдання №8 Results Evaluation and Release

- Деталі завдання: захистити звіт по 3 епіку та вивчений матеріал.

2. Дизайн та планована оцінка часу виконання завдань:

Програма №1 Lab# programming: Lab 2

- Блок-схема:

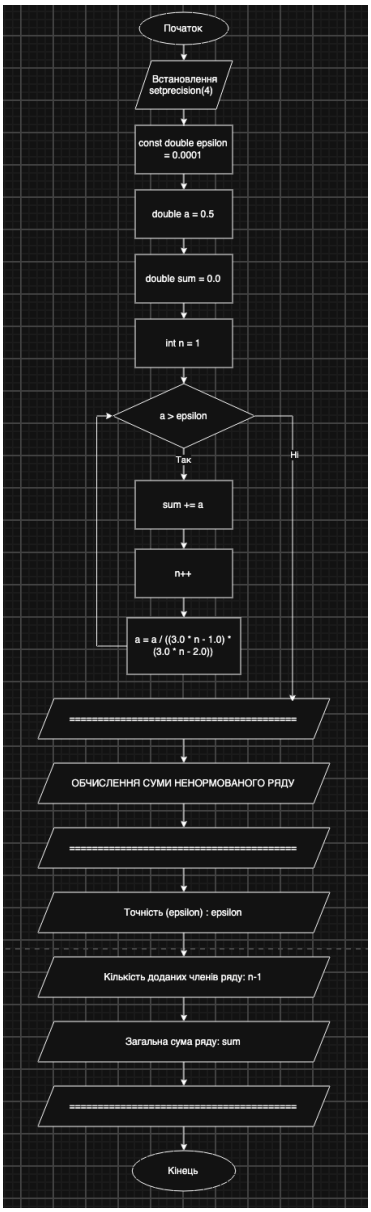


Рис 7. Блок-схема до задачі lab 2

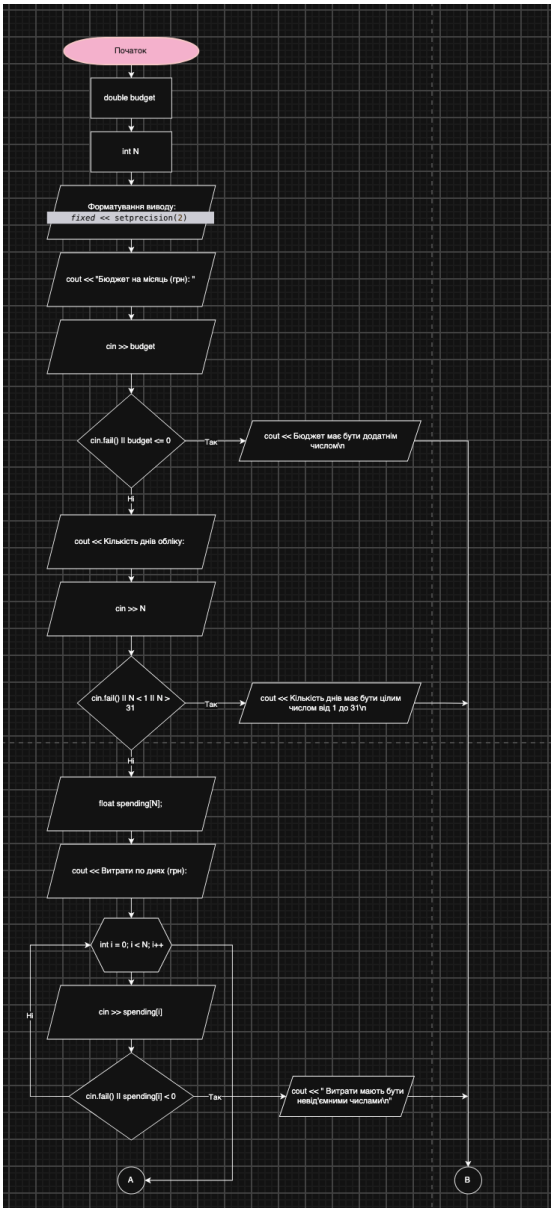
- Планований час на реалізацію: 35хв

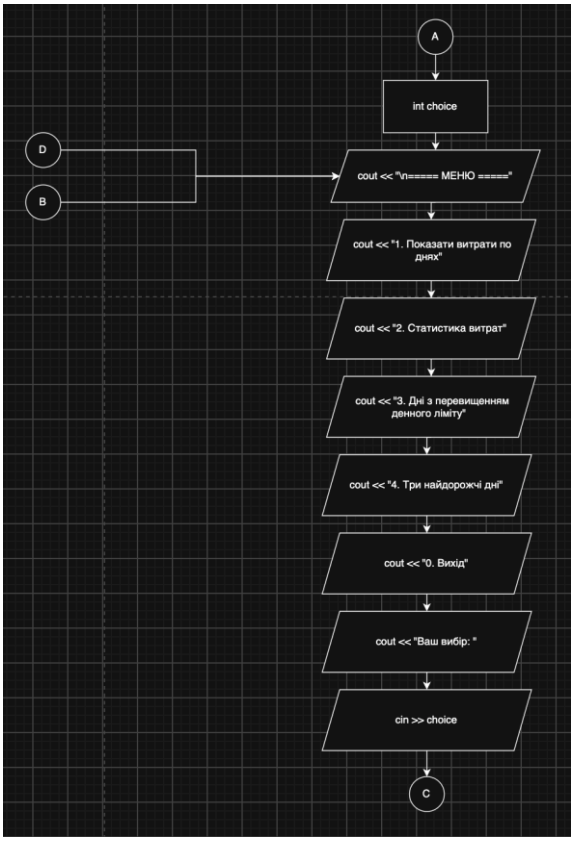
Програма №2 Lab# programming: Lab 3

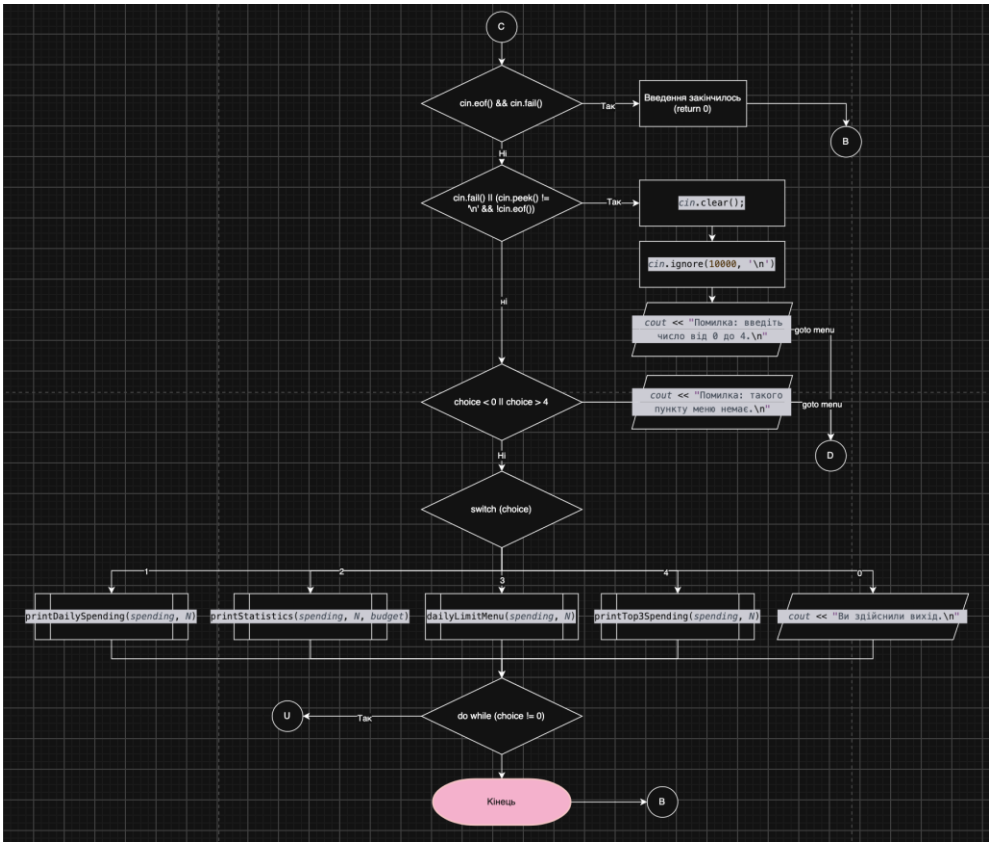
- Планований час на реалізацію: 40 хв

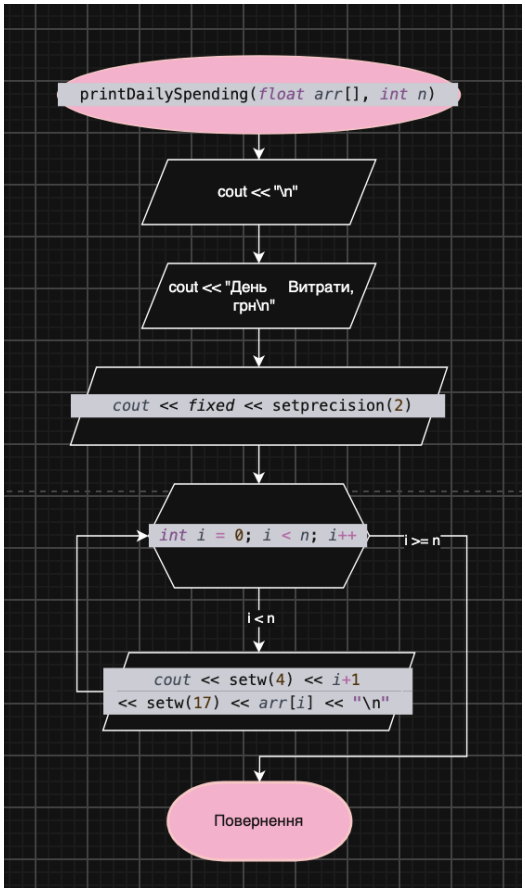
Програма 3 Practice# programming: Class Practice Task

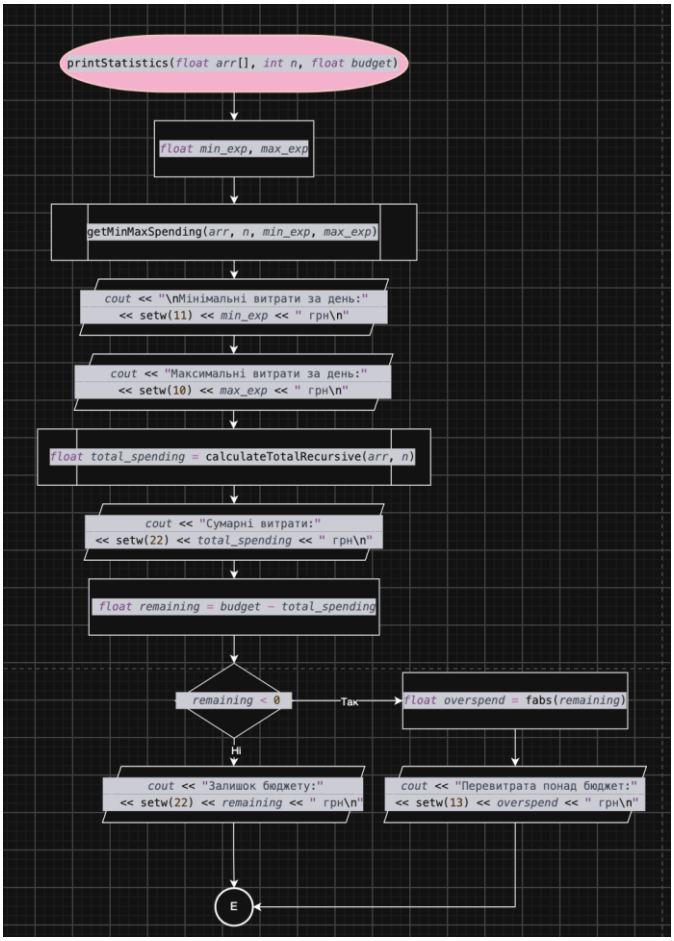
- Блок-схема:

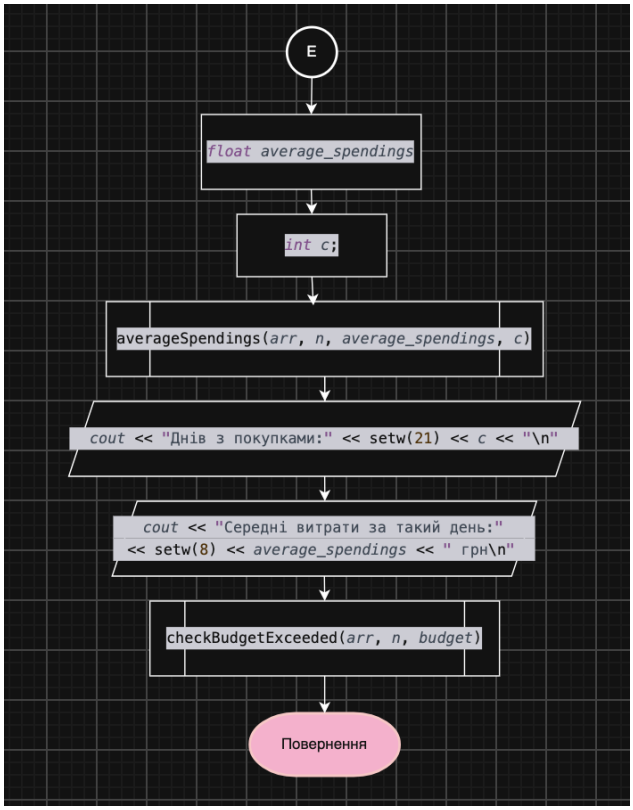


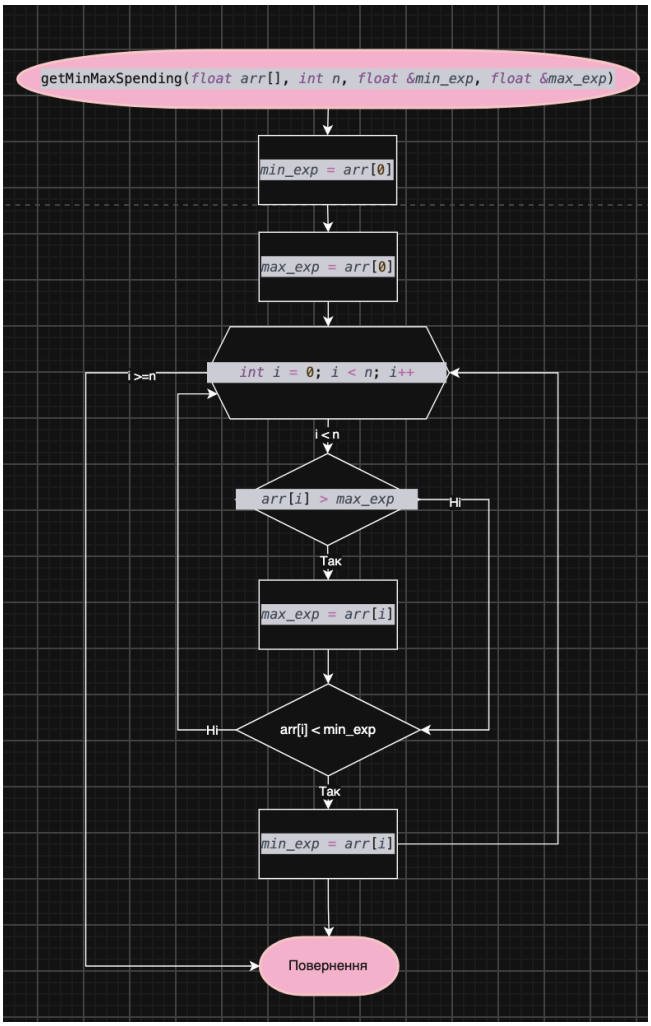

A
int choice
D
B
cout << "\n===== МЕНЮ =====
cout << "1. Показати витрати по днях"
cout << "2. Статистика витрат"
cout << "3. Дні з перевищенням денного ліміту"
cout << "4. Три найдорожчі дні"
cout << "0. Вихід"
cout << "Ваш вибір: "
cin >> choice
C

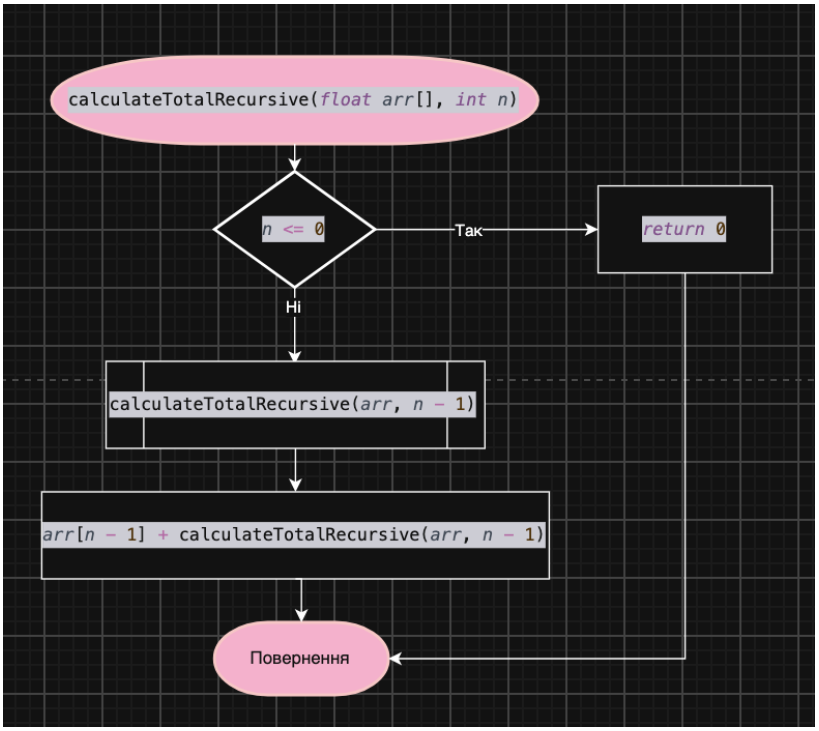

C
cin.eof() && cin.fail()
Так
Введення закінчилось (return 0)
B
Ні
cin.fail() || (cin.peek() != '\n' && !cin.eof())
Так
cin.clear();
cin.ignore(10000, '\n')
Ні
cout << "Помилка: введіть число від 0 до 4.\n"
goto menu
choice < 0 || choice > 4
cout << "Помилка: такого пункту меню немає.\n"
goto menu
D
Ні
switch (choice)
1
2
3
4
0
printDailySpending(spending, N)
printStatistics(spending, N, budget)
dailyLimitMenu(spending, N)
printTop3Spending(spending, N)
cout << "Ви здійснили вихід.\n"
U
Так
do while (choice != 0)
Кінець
B

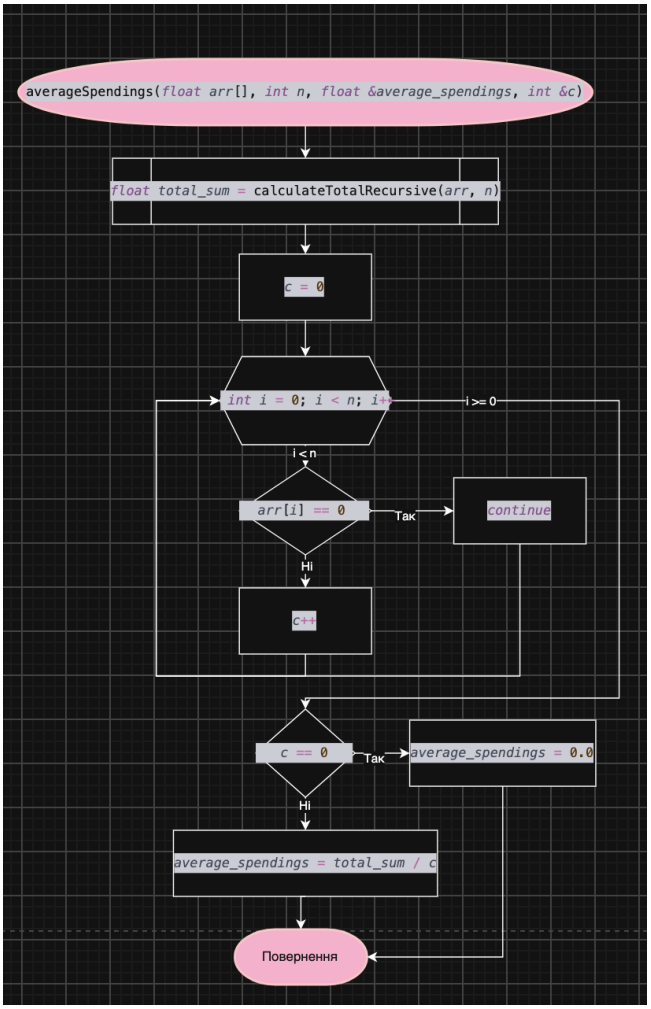

printDailySpending(float arr[], int n)
cout << "\n"
cout << "День Витрати, грн\n"
cout << fixed << setprecision(2)
int i = 0; i < n; i++
i >= n
i < n
cout << setw(4) << i+1 << setw(17) << arr[i] << "\n"
Повернення

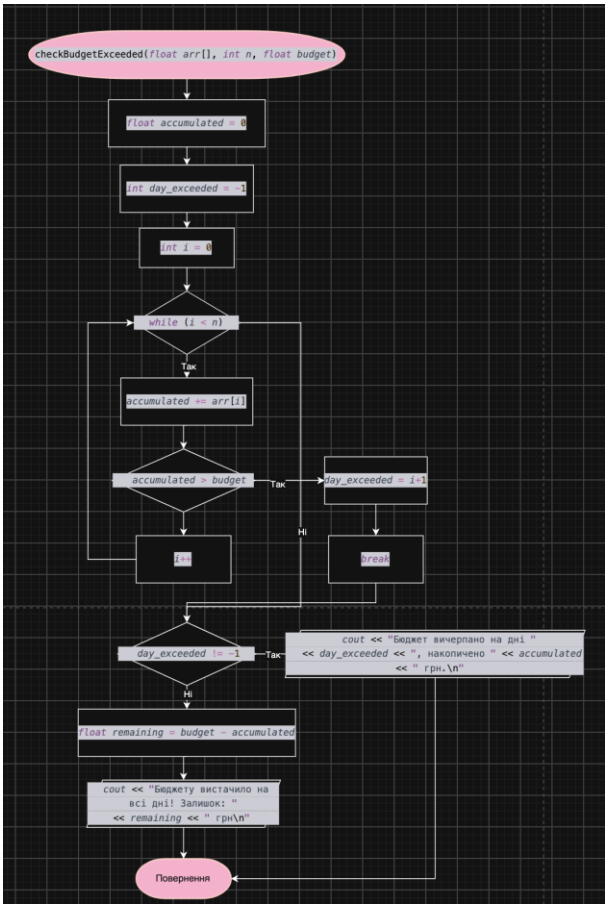

printStatistics(float arr[], int n, float budget)
float min_exp, max_exp
getMinMaxSpending(arr, n, min_exp, max_exp)
cout << "\nМінімальні витрати за день:" << setw(11) << min_exp << " грн\n"
cout << "Максимальні витрати за день:" << setw(10) << max_exp << " грн\n"
float total_spending = calculateTotalRecursive(arr, n)
cout << "Сумарні витрати:" << setw(22) << total_spending << " грн\n"
float remaining = budget - total_spending
remaining < 0
Так
float overspend = fabs(remaining)
Ні
cout << "Залишок бюджету:" << setw(22) << remaining << " грн\n"
cout << "Перевитрата понад бюджет:" << setw(13) << overspend << " грн\n"
E

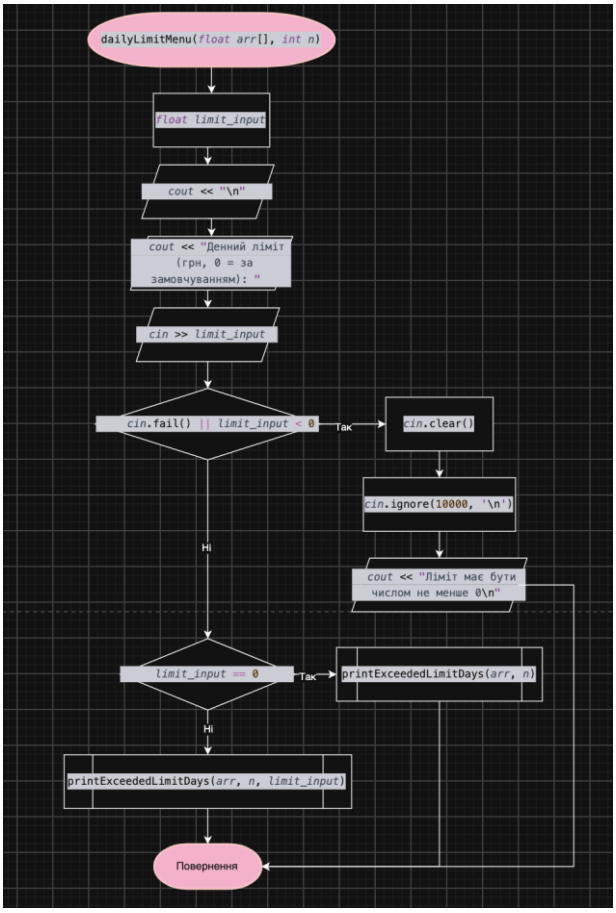

E
float average_spendings
int c;
averageSpendings(arr, n, average_spendings, c)
cout << "Днів з покупками:" << setw(21) << c << "\n"
cout << "Середні витрати за такий день:"
<< setw(8) << average_spendings << " грн\n"
checkBudgetExceeded(arr, n, budget)
Повернення

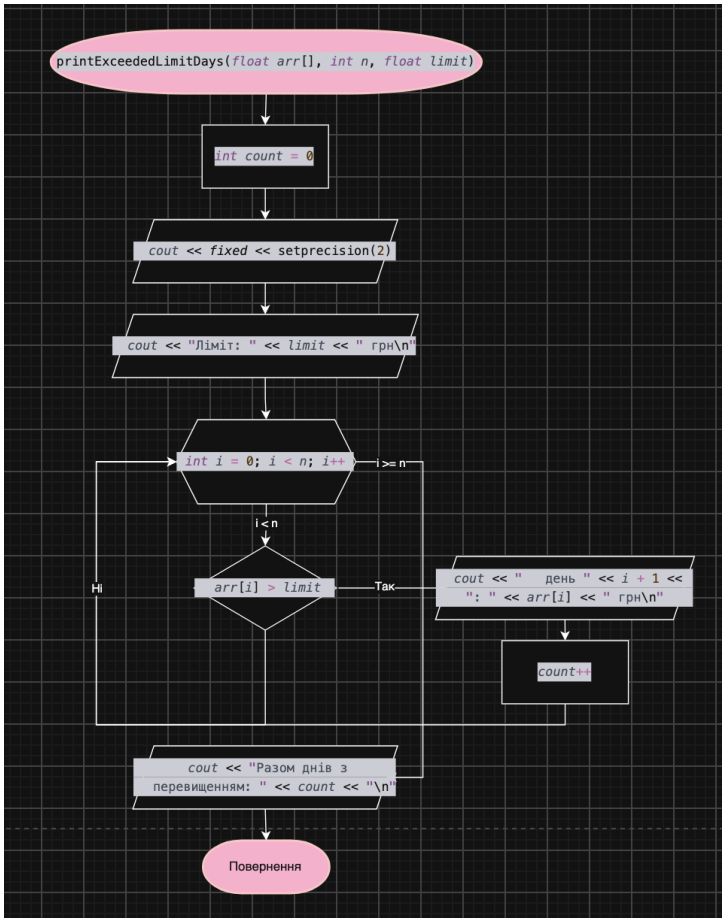

getMinMaxSpending(float arr[], int n, float &min_exp, float &max_exp)
min_exp = arr[0]
max_exp = arr[0]
int i = 0; i < n; i++
i >=n
i < n
arr[i] > max_exp
Ні
Так
max_exp = arr[i]
Ні
arr[i] < min_exp
Так
min_exp = arr[i]
Повернення

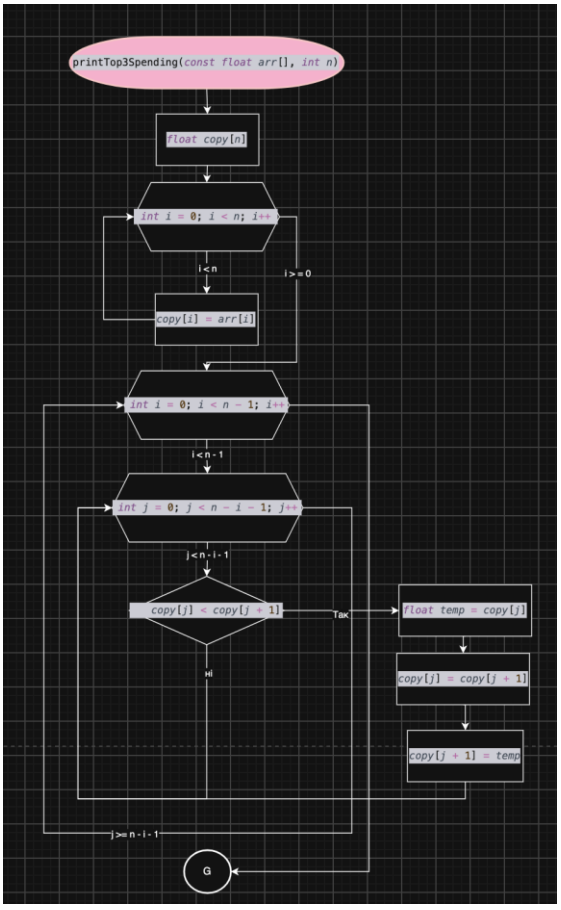

calculateTotalRecursive(float arr[], int n)
n <= 0
Так
return 0
Ні
calculateTotalRecursive(arr, n - 1)
arr[n - 1] + calculateTotalRecursive(arr, n - 1)
Повернення


averageSpendings(float arr[], int n, float &average_spendings, int &c)
float total_sum = calculateTotalRecursive(arr, n)
c = 0
int i = 0; i < n; i++
i >= 0
i < n
arr[i] == 0
Так
continue
Ні
c++
c == 0
Так
average_spendings = 0.0
Ні
average_spendings = total_sum / c
Повернення


checkBudgetExceeded(float arr[], int n, float budget)
float accumulated = 0
int day_exceeded = -1
int i = 0
while (i < n)
Так
accumulated += arr[i]
accumulated > budget
Так
day_exceeded = i+1
Ні
i++
break
day_exceeded != -1
Так
cout << "Бюджет вичерпано на дні "
<< day_exceeded << ", накопичено " << accumulated
<< " грн.\n"
Ні
float remaining = budget - accumulated
cout << "Бюджету вистачило на
всі дні! Залишок: "
<< remaining << " грн\n"
Повернення


dailyLimitMenu(float arr[], int n)
float limit_input
cout << "\n"
cout << "Денний ліміт
(грн, 0 = за
замовчуванням): "
cin >> limit_input
cin.fail() || limit_input < 0
Так
cin.clear()
cin.ignore(10000, '\n')
cout << "Ліміт має бути
числом не менше 0\n"
Ні
limit_input == 0
Так
printExceededLimitDays(arr, n)
Ні
printExceededLimitDays(arr, n, limit_input)
Повернення


printExceededLimitDays(float arr[], int n, float limit)
int count = 0
cout << fixed << setprecision(2)
cout << "Ліміт: " << limit << " грн\n"
int i = 0; i < n; i++
i >= n
i < n
arr[i] > limit
Ні
Так
cout << " день " << i + 1 << ": " << arr[i] << " грн\n"
count++
cout << "Разом днів з перевищенням: " << count << "\n"
Повернення


printTop3Spending(const float arr[], int n)
float copy[n]
int i = 0; i < n; i++
i < n
i >= 0
copy[i] = arr[i]
int i = 0; i < n - 1; i++
i < n - 1
int j = 0; j < n - i - 1; j++
j < n - i - 1
copy[j] < copy[j + 1]
Так
Ні
float temp = copy[j]
copy[j] = copy[j + 1]
copy[j + 1] = temp
j >= n - i - 1
G

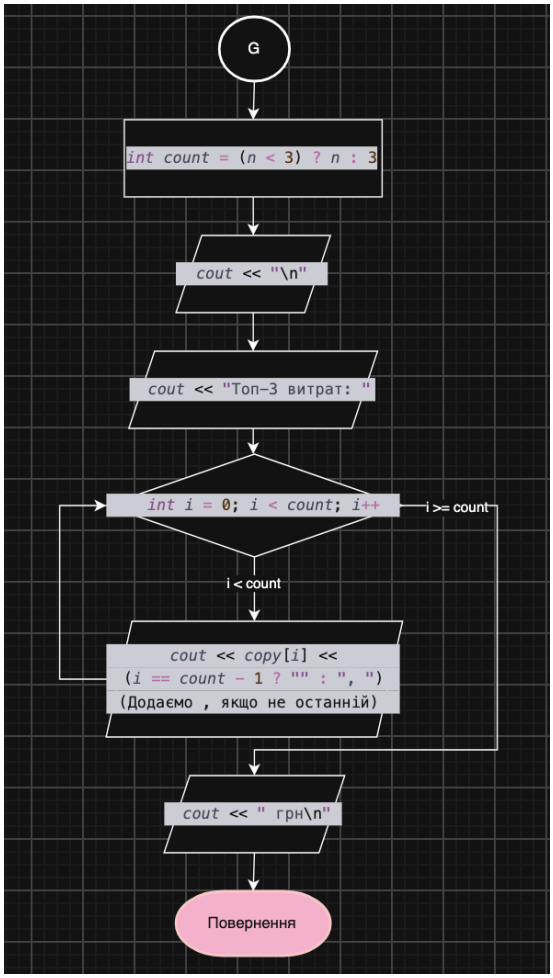


Рис 8-21 . Блок-схема до задачі Class Practice task

- Планований час на реалізацію: 50 хв

Програма №4 Practice# programming: Self Practice Task 1

- Планований час на реалізацію: 20 хв

Програма №5 Practice# programming: Self Practice Task 2

- Планований час на реалізацію: 20 хв

Програма №6 Practice# programming: Self Practice Task 3

- Планований час на реалізацію: 20 хв

4. Код програм з посиланням на зовнішні ресурси:

Завдання №1 Lab# programming: Lab 2

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << fixed << setprecision(4);

    const double epsilon = 0.0001;
    double a = 0.5;
    double sum = 0.0;
    int n = 1;

    while (a > epsilon) {
        sum += a;
        n++;
        a = a / ((3.0 * n - 1.0) * (3.0 * n - 2.0));
    }

    cout << "=======================================\n";
    cout << "   ОБЧИСЛЕННЯ СУМИ НЕНОРМОВАНОГО РЯДУ  \n";
    cout << "=======================================\n";
    cout << "Точність (epsilon): " << epsilon << endl;
    cout << "Кількість доданих членів ряду: " << n - 1 << endl;
    cout << "Загальна сума ряду: " << sum << endl;
    cout << "=======================================\n";
    return 0;
}
```

Рис 22. код до задачі lab 2

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/lab_2_task_1_variant_13_dariia_sierova.cpp

Завдання №2 Lab# programming: Lab 3

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>

using namespace std;

int main() {
    const double a = 0.1;
    const double b = 0.8;
    const int k = 10;
    const int n = 3;
    const double epsilon = 0.0001;

    const double step = (b - a) / k;

    cout << "=====================================================" << endl;
    cout << "                  Обчислення функції" << endl;
    cout << "=====================================================" << endl;
    cout << fixed << setprecision(5);

    for (int j = 0; j <= k; ++j) {
        // Поточне значення x на кожному кроці
        const double x = a + j * step;

        const double a_0 = x; // Перший доданок ряду (при i = 0)
        const double x4 = x * x * x * x;

        // 1. Обчислення SN (n = 3)
        double SN = a_0;
        double a_i = a_0;
        for (int i = 1; i <= n; ++i) {
            // Обчислюємо наступний доданок через попередній
            a_i = a_i * x4 * (4.0 * i - 3.0) / (4.0 * i + 1.0);
            SN += a_i;
        }

        // 2. Обчислення SE (epsilon)
        double SE = 0.0;
        a_i = a_0; //Скидаємо змінну a_i назад до першого значення a_0 = x
        int i = 1;
        while (fabs(a_i) >= epsilon) {
            SE += a_i; // Додаємо тільки якщо |a_i| >= epsilon
            a_i = a_i * x4 * (4.0 * i - 3.0) / (4.0 * i + 1.0);
            ++i;
        }

        // 3. Точне значення функції Y
        const double Y = 0.25 * log((1.0 + x) / (1.0 - x)) + 0.5 * atan(x);

        cout << "X=" << setw(8) << x
             << " SN=" << setw(9) << SN
             << " SE=" << setw(9) << SE
             << " Y=" << setw(9) << Y << endl;
    }

    cout << "=====================================================" << endl;
    return 0;
}
```

Рис 23-24. код до задачі lab 3

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/lab_3_task_1_variant_9_dariia_sierova.cpp

Завдання №3 Lab# programming: Class Practice Task

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>

using namespace std;

// Прототипи функцій
void printDailySpending(float arr[], int n);
void getMinMaxSpending(float arr[], int n, float &min_exp, float &max_exp);
float calculateTotalRecursive(float arr[], int n);
void averageSpendings(float arr[], int n, float &average_spendings, int &c);
void checkBudgetExceeded(float arr[], int n, float budget);
void printExceededLimitDays(float arr[], int n, float limit = 1000.0f);
void printTop3Spending(const float arr[], int n);
void printStatistics(float arr[], int n, float budget);
void dailyLimitMenu(float arr[], int n);

int main(){

    float budget;
    int N;

    cout << fixed << setprecision(2);

    cout << "Бюджет на місяць (грн): ";
    cin >> budget;
    if (cin.fail() || budget <= 0 ) {
        cout << "Бюджет має бути додатнім числом\n";
        return 1;
    }

    cout << "Кількість днів обліку: ";
    cin >> N;
    if (cin.fail() || N < 1 || N > 31){
        cout << "Кількість днів має бути цілим числом від 1 до 31\n";
        return 1;
    }

    float spending[N];

    cout << "Витрати по днях (грн): ";
    for (int i = 0; i < N; i++){
        cin >> spending[i];

        if (cin.fail() || spending[i] < 0){
            cout << " Витрати мають бути невід'ємними числами\n";
            return 1;
        }
    }
```

```cpp
    int choice;

    do {
    menu: // мітка, на яку повертає goto
        cout << "\n===== МЕНЮ =====\n";
        cout << "1. Показати витрати по днях\n";
        cout << "2. Статистика витрат\n";
        cout << "3. Дні з перевищенням денного ліміту\n";
        cout << "4. Три найдорожчі дні\n";
        cout << "0. Вихід\n";
        cout << "Ваш вибір: ";
        cin >> choice;

        if (cin.eof() && cin.fail()) return 0; // введення закінчилось

        // Якщо нечислове введення (або зайві символи після числа), то назад до меню
        if (cin.fail() || (cin.peek() != '\n' && !cin.eof())) {
            cin.clear();               // скидаємо помилку
            cin.ignore(10000, '\n');   // викидаємо неправильні символи, щоб не було зациклення
            cout << "Помилка: введіть число від 0 до 4.\n";
            goto menu;
        }

        // Якщо введено неіснуючий пункт меню, то назад до меню
        if (choice < 0 || choice > 4) {
            cout << "Помилка: такого пункту меню немає.\n";
            goto menu;
        }

        switch (choice) {
            case 1: printDailySpending(spending, N); break;
            case 2: printStatistics(spending, N, budget); break;
            case 3: dailyLimitMenu(spending, N); break;
            case 4: printTop3Spending(spending, N); break;
            case 0: cout << "Ви здійснили вихід.\n"; break;
        }

    } while (choice != 0);

    return 0;
}
```

```cpp
// 1: Показати витрати по днях
void printDailySpending(float arr[], int n){

    cout << "\nДень     Витрати, грн\n";

    cout << fixed << setprecision(2);
    for (int i = 0; i < n; i++){
        cout << setw(4) << i+1 << setw(17) << arr[i] << "\n";
    }

}

// 2: Статистика витрат
void printStatistics(float arr[], int n, float budget){

    // Вивід мінімальних та максимальних витрат за день
    float min_exp, max_exp;
    getMinMaxSpending(arr, n, min_exp, max_exp);

    cout << "\nМінімальні витрати за день:" << setw(11) << min_exp << " грн\n";
    cout << "Максимальні витрати за день:" << setw(10) << max_exp << " грн\n";

    // Вивід загальних витрат
    float total_spending = calculateTotalRecursive(arr, n);
    cout << "Сумарні витрати:" << setw(22) << total_spending << " грн\n";

    // Перевірка залишку бюджету
    float remaining = budget - total_spending;

    if (remaining < 0) {
        float overspend = fabs(remaining); // модуль від'ємного залишку
        cout << "Перевитрата понад бюджет:" << setw(13) << overspend << " грн\n";
    } else {
        cout << "Залишок бюджету:" << setw(22) << remaining << " грн\n";
    }

    // Обчислення середніх витрат за дні з покупками
    float average_spendings;
    int c;
    averageSpendings(arr, n, average_spendings, c);

    cout << "Днів з покупками:" << setw(21) << c << "\n";
    cout << "Середні витрати за такий день:" << setw(8) << average_spendings << " грн\n";

    // Перевірка дня, коли бюджет вичерпано
    checkBudgetExceeded(arr, n, budget);
}
```

```
// Пошук мін та макс витраченої суми за день
void getMinMaxSpending(float arr[], int n, float &min_exp, float &max_exp){
    min_exp = arr[0];
    max_exp = arr[0];

    for (int i = 0; i < n; i++){
        if (arr[i] > max_exp) max_exp = arr[i];
        if (arr[i] < min_exp) min_exp = arr[i];
    }
}

// Обчислення сумарних витрат рекурсивно
float calculateTotalRecursive(float arr[], int n){
    if (n <= 0) return 0;

    return arr[n - 1] + calculateTotalRecursive(arr, n - 1);
}

// Обчислення середніх витрат за дні з покупками
void averageSpendings(float arr[], int n, float &average_spendings, int &c){

    float total_sum = calculateTotalRecursive(arr, n);

    c = 0;
    for (int i = 0; i < n; i++){
        if (arr[i] == 0 ) continue;
        else c++;
    }
    if (c == 0) average_spendings = 0.0;
    else average_spendings = total_sum / c;

}

// Перевірка дня, коли бюджет вичерпано
void checkBudgetExceeded(float arr[], int n, float budget){

    float accumulated = 0;
    int day_exceeded = -1; // означає не перевищено

    int i = 0;
    while (i < n){

        accumulated += arr[i];

        if (accumulated > budget){
            day_exceeded = i+1;
            break;
        }
        i++;
    }
```

```
    if (day_exceeded != -1){
        cout << "Бюджет вичерпано на дні " << day_exceeded << ", накопичено " << accumulated << "
    }
    else {
        float remaining = budget - accumulated;
        cout << "Бюджету вистачило на всі дні! Залишок: " << remaining << " грн\n";
    }
}

// 3: Дні з перевищенням денного ліміту
void dailyLimitMenu(float arr[], int n){
    float limit_input;

    cout << "\nДенний ліміт (грн, 0 = за замовчуванням): ";
    cin >> limit_input;

    // Валідація: число, не менше 0. Інакше повертаємось у меню
    if (cin.fail() || limit_input < 0) {
        cin.clear();
        cin.ignore(10000, '\n');
        cout << "Ліміт має бути числом не менше 0\n";
        return;
    }

    if (limit_input == 0) {
        printExceededLimitDays(arr, n);
    } else {
        printExceededLimitDays(arr, n, limit_input);
    }
}

void printExceededLimitDays(float arr[], int n, float limit){
    int count = 0;

    cout << fixed << setprecision(2);
    cout << "Ліміт: " << limit << " грн\n";

    for (int i = 0; i < n; i++) {
        if (arr[i] > limit) {
            cout << "   день " << i + 1 << ": " << arr[i] << " грн\n";
            count++;
        }
    }

    cout << "Разом днів з перевищенням: " << count << "\n";
}
```

```cpp
//4: Три найдорожчі дні
void printTop3Spending(const float arr[], int n) {

    // Створення копії масиву
    float copy[n];
    for (int i = 0; i < n; i++) {
        copy[i] = arr[i];
    }

    // Сортування копії за спаданням (метод бульбашки)
    for (int i = 0; i < n - 1; i++){
        for (int j = 0; j < n - i - 1; j++){
            if (copy[j] < copy[j + 1]) {
                float temp = copy[j];
                copy[j] = copy[j + 1];
                copy[j + 1] = temp;
            }
        }
    }

    int count = (n < 3) ? n : 3; //Якщо n менше 3 — візьми n (? n), інакше візьми 3 (: 3)

    cout << "\nТоп-3 витрат: ";
    for (int i = 0; i < count; i++){
        cout << copy[i] << (i == count - 1 ? "" : ", ");
    }
    /* i == count - 1 — перевірка: «Чи є поточний елемент ОСТАННІМ у списку?»
? "" — якщо це останній елемент, виводиться порожній рядок (нічого не додається).
: ", " — якщо елемент НЕ останній, після числа виводиться кома з пробілом. */
    cout << " грн\n";
}
```

Рис 25-30. код до Class Practice task

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/practice_task_1_dariia_sierova.cpp

Завдання №4 Practice# programming: Self Practice Task 1

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a;
    cin >> b;
    cin >> c;
    if ( (a + b) > c ) {
        cout << "YES\n";
    } else {
        cout << "NO";
    }

    return 0;
}
```

Рис 31. Код до задачі Self Practice Task 1

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/self_practice_algotester_task_1_dariia_sierova.cpp

Завдання №5 Practice# programming: Self Practice Task 2

```cpp
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    if (n >= 1 && n <=100 && m>= 1 && m <= 100){

        if ((n * m) % 2 == 1) {
            cout << "Imp" << endl;
        } else {
            cout << "Dragon" << endl;
        }
    }
    return 0;
}
```

Рис 32. Код до задачі Self Practice Task 2

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/self_practice_algotester_task_2_dariia_sierova.cpp

Завдання №6 Practice# programming: Self Practice Task 3

```cpp
#include <iostream>
using namespace std;

void findMax(int num[], int n) {
    int max = num[0];

    for (int i = 1; i < n; ++i) {
        if (num[i] > max) {
            max = num[i];
        }
    }
    cout << max << endl;
}

int main() {
    int n;
    cin >> n;

    if (n > 0 && n < 1000) {
        int num[n];
        for (int i = 0; i < n; ++i) {
            cin >> num[i];
        }
        findMax(num, n);
    }
    return 0;
}
```

Рис 33. Код до задачі Self Practice Task 3

Посилання на файл коду в пулреквесті: https://github.com/artificial-intelligence-department/ai_programming_playground_2026/blob/28d0b219d2347cbe51f4784d3fd882f3f3e44215/ai_13/daria_sierova/epic_3/self_practice_algotester_task_3_dariia_sierova.cpp

5. Результати виконання завдань, тестування та фактично затрачений час:

Завдання №1 Lab# programming: Lab 2

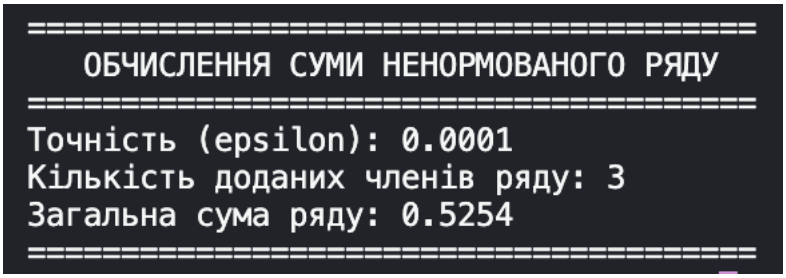


Рис 34. Виконання програми lab 2

Час затрачений на виконання завдання: 25хв

Завдання №2 Lab# programming: Lab 3



Рис 35. Виконання програми lab 3

Час затрачений на виконання завдання: 40хв

Завдання №3 Practice# programming: Class Practice Task

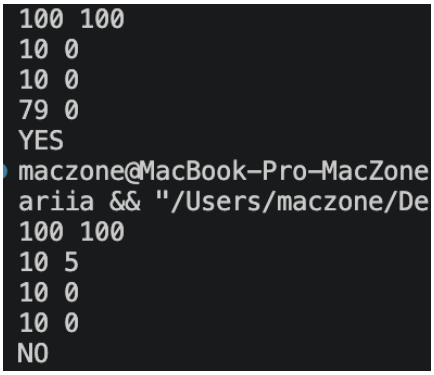


Рис 36. Виконання програми Class Practice task

Час затрачений на виконання завдання: 5 год

Завдання №4 Practice# programming: Self Practice Task 1



Рис 37. Виконання програми self practice task 1

Час затрачений на виконання завдання: 10хв

Завдання №5 Practice# programming: Self Practice Task 2

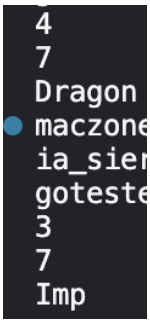


Рис 38. Виконання програми self practice task 2

Час затрачений на виконання завдання: 15хв

Завдання №6 Practice# programming: Self Practice Task 3

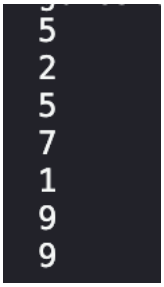


Рис 39. Виконання програми self practice task 3

Час затрачений на виконання завдання: 20хв

6. Кооперація з командою:

- Скрін з 1-ї зустрічі по обговоренню задач Епіку та Скрін прогресу по Трелло



Рис 40. Зустріч з командою 1

- Скрін з 2-ї зустрічі по обговоренню задач Епіку та Скрін прогресу по Трелло

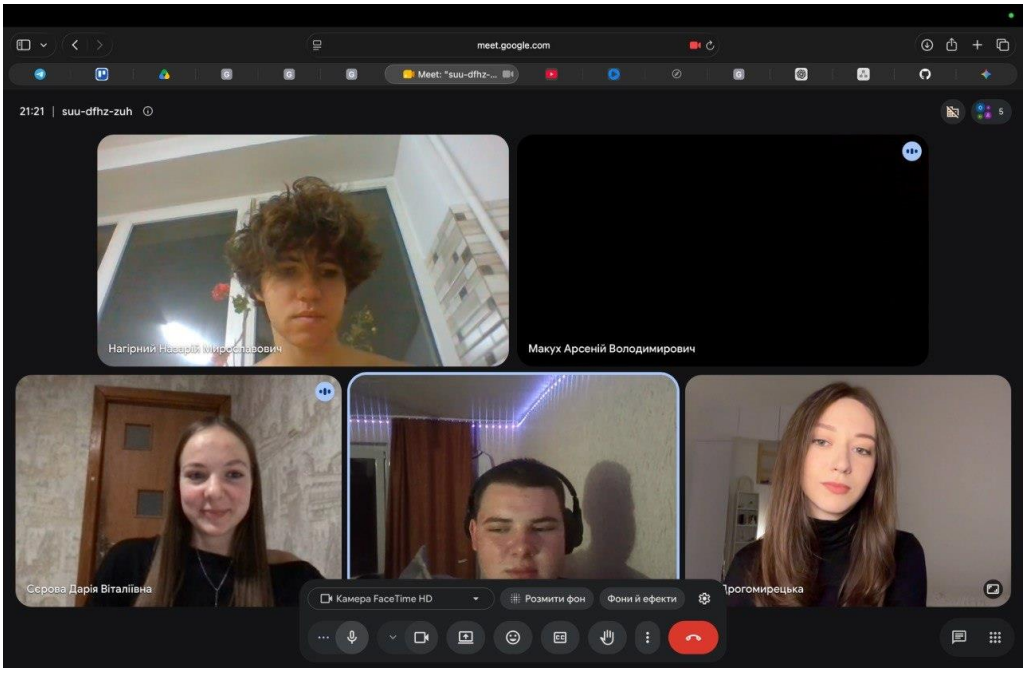


Рис 41. Зустріч з командою 2

- Скрін командної дошки Трелло

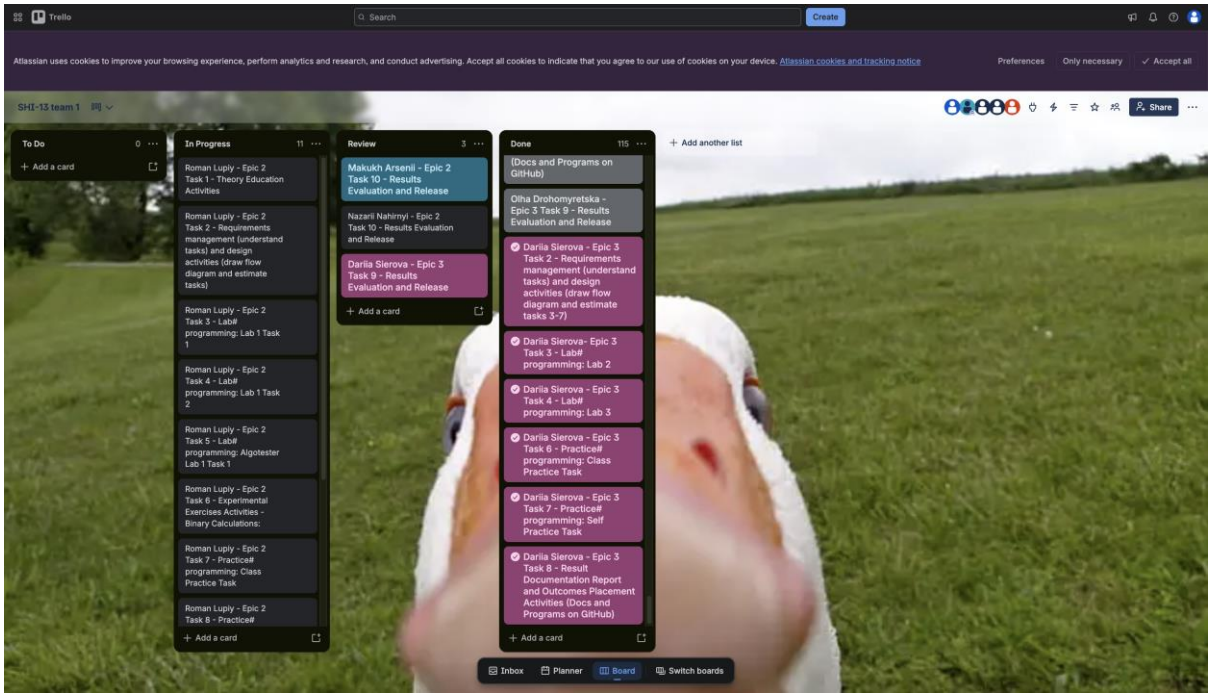


Рис 42. Дошка Трелло

Висновки:

У ході виконання роботи було систематизовано та поглиблено теоретичні знання й практичні навички роботи з мовою програмування С++.

Під час опрацювання матеріалу було:

1. **Засвоєно базові та вкладені циклічні конструкції** (while, do-while, for) та механізми керування ними (break, continue), що дозволило ефективно організовувати повторювані та ітераційні обчислювальні процеси.
2. **Опановано методи роботи з одновимірними масивами**, зокрема алгоритми їх обходу, пошуку екстремумів (мінімуму/максимуму), обчислення агрегованих значень та виконання базових алгоритмів сортування.
3. **Реалізовано принципи модульного програмування** за допомогою функцій: засвоєно механізми передачі аргументів за значенням та за посиланням, правила області видимості змінних, принципи побудови рекурсивних алгоритмів та застосування специфікатора inline разом із вбудованими математичними бібліотеками (<cmath>, <algorithm>).

Отримані знання дозволяють проектувати структурований, оптимізований та читабельний модульний код для розв'язання складних обчислювальних задач, а також будувати відповідні алгоритми й оформлювати результати розробки відповідно до сучасних вимог.